

Payment Failure Recovery Engine

Razorpay Hackathon — Project Brief & Build Plan

Track: AI Revenue Recovery | Format: GitHub repo → 5-min pitch → architecture walkthrough → panel interview | Stipend: ₹75k/month | Location: Bangalore (in-person) | Apps close: 5 September

Indian checkout success rates sit in the high 80s. Every failed payment is real, recoverable money. Most merchants retry on a dumb fixed schedule and burn customer goodwill in the process. This project decides whether, when, and on which rail to retry a failed payment — and proves the improvement with numbers, not vibes.

Why This Approach, Not a Generic AI Demo

The panel round is an architecture interview, not a demo reel — a judge will ask why each decision was made, so depth beats surface area. Revenue Recovery is also the most under-built track relative to how measurable and unglamorous the underlying problem is, which is exactly what Razorpay actually pays people to solve.

What separates this from ~2,000 other submissions:

- **Real infrastructure, not a mock.** Real Razorpay test-mode webhooks, real API calls, idempotency keys on every retry attempt. A mocked gateway reads as a toy.
- **An eval harness with a results table.** Baseline vs. agent, across N runs, with variance shown. Almost nobody does this — it is the single loudest signal in the whole submission.
- **Deterministic gates before the LLM.** No LLM is ever allowed to directly authorize money movement. That is a fintech panel's first question — the README should answer it before they ask.
- **Documented failure cases.** Showing exactly where the agent breaks reads as senior engineering judgment. Hiding failure cases reads as junior.

Architecture — Five Layers, In This Order

Order matters here: deterministic logic wraps the LLM on both sides, so the model only ever chooses from a constrained action space and never executes anything directly.

Layer	What it does
1. Ingestion	Razorpay test-mode webhooks (payment.failed, payment.captured), signature verification, idempotent handlers, all events logged to Postgres and fully replayable.
2. Deterministic classifier	Maps gateway error code → failure taxonomy: insufficient funds, issuer decline, 3DS drop-off, bank downtime, UPI collect timeout, expired instrument. No LLM here — a regex-solvable problem solved by an LLM is the first thing a panel will flag.
3. Policy agent (LLM)	Inputs: failure class, customer retry history, bank, amount, time of day. Output: a constrained JSON action from a fixed set — retry now / retry at T / switch rail / abandon / nudge customer. Constrained, never freeform.
4. Guardrail gate	Deterministic, runs before execution: hard-decline blocklist, max retries per instrument, amount ceiling, consent window, per-customer rate limit, idempotency key on every attempt. Any agent output that fails schema or policy is rejected and logged. This is the answer to "what stops it from doing something stupid with real money."

5. Recovery messaging (LLM)	Generates the plain-language customer nudge explaining the failure and next step — a scoped, legitimate use of generation rather than a decision-making one.
------------------------------------	--

The Part That Actually Gets You Selected: The Eval Harness

Build a bank-response simulator with per-rail, per-bank, per-hour success probabilities. Replay roughly 5,000 synthetic failures through three policies — no retry, a fixed 3-retry baseline, and the agent — and report recovery rate, retry cost, false-retry rate, and time-to-recovery, across multiple random seeds with variance shown. State plainly that the bank model is synthetic; naming that assumption is a maturity signal, not a weakness.

Headline metric
“■X recovered per ■1Cr of failed volume vs. baseline, at Y% fewer retry attempts.” That single number is the whole pitch — build the eval before the agent, because it forces honest numbers.

Two-Week Build Shape

~2 weeks to the 5 September deadline. Notice the eval harness (days 4–6) comes before the policy agent (days 7–10) — that ordering is deliberate.

Days	Focus
1–3	Webhook ingestion + event store + deterministic classifier
4–6	Bank-response simulator + baseline policies (eval before agent)
7–10	Policy agent (LLM) + guardrail gate
11–12	Dashboard: recovery funnel, per-bank breakdown
13–14	README with architecture diagram, one-command run, documented failure cases, 3-min demo video

On Reusing the Existing commerce-os Pipeline

The existing governance pipeline is the right structural skeleton for this — the deterministic-gate pattern transfers directly. But resubmitting it as-is will read as a reheated hackathon project. Strip the deterministic-gate layer out and rebuild it specifically around this one Razorpay problem.

Other Track Options, For Reference

Risk Manager	Merchant onboarding risk scoring — crawl the merchant site, classify against prohibited-business categories, emit a decision with an evidence trail and a confidence-based escalation queue. Razorpay already runs something like this internally, so the audit trail needs to be built well.
Finance Controller	Settlement reconciliation — match settlement files against a ledger, explain every mismatch in plain language, and auto-draft journal entries.

No project is a 100% lock — the panel round is partly about how well you think on your feet. But of the five tracks, this is the one worth betting on: it is specified tightly enough to start building tonight.

Prepared as a working brief for hackathon build planning.